The Complete Guide to XGBoost (Extreme Gradient Boosting) — 2026 Edition

Current as of XGBoost 3.3.0 (released 17 June 2026). All code targets the 2.x/3.x API.

TL;DR

- **What XGBoost really is:** a regularized, second-order (Newton) gradient-boosted tree system. Every design choice — the leaf-weight formula `w* = −G/(H+λ)`, the split-gain formula, sparsity-aware default directions, the weighted quantile sketch — falls out of one idea: minimize `loss + Ω` using a **second-order Taylor expansion** where each data point contributes only a gradient g_i and a Hessian h_i. Learn that expansion and the rest of the library becomes readable.
- **What to actually do:** Use the scikit-learn API with `tree_method="hist"` (the default since 2.0), enable `early_stopping_rounds` with an `eval_set`, tune in the order *learning rate/trees → tree structure → sampling → regularization*, interpret with **SHAP** rather than raw `gain`/`weight` importances, and handle imbalance with `scale_pos_weight` + AUC-PR. XGBoost remains the best-tuned, most reliable default for tabular data.
- **When to pick something else:** For very large or highly sparse data where wall-clock time dominates, LightGBM is materially faster (the LightGBM authors report per-iteration speedups of "roughly 2× (LETOR) to 9.4× (Allstate)" over XGBoost's histogram mode, and XGBoost ran *out of memory* on their KDD10/KDD12 benchmarks where LightGBM trained fine). For datasets dominated by high-cardinality categoricals with minimal tuning, CatBoost is often the better out-of-the-box choice. For most other tabular problems, XGBoost wins on tunability and ecosystem.

Part 1 — The Math Behind XGBoost, Step by Step

1.1 The model: an additive ensemble of CARTs

XGBoost predicts by summing the outputs of K classification-and-regression trees (CARTs), each mapping an input to a real-valued leaf score:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F}$$

where $\mathcal{F}$ is the space of CARTs. A single tree is written `f(x) = w_{q(x)}`, where `q: ℝᵈ → {1,…,T}` maps a sample to one of T leaves and `w ∈ ℝ^T` is the vector of leaf scores. (Random forests use the *same* model class; the difference is purely how the trees are trained.) readthedocs

1.2 The regularized objective

XGBoost's objective always has two parts — training loss plus a complexity penalty:

$$\text{obj}(\theta) = \underbrace{\sum_{i=1}^{n} l(y_i, \hat{y}_i)}_{\text{training loss}} + \underbrace{\sum_{k=1}^{K} \Omega(f_k)}_{\text{regularization}}$$

The per-tree complexity is defined explicitly (this is the "eXtreme" in XGBoost — most classic GBM implementations left complexity control to heuristics): readthedocs

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

- `γ` (`gamma`, a.k.a. `min_split_loss`) penalizes the **number of leaves** `T` — a per-leaf cost that must be "paid" by a split.
- `λ` (`lambda`, `reg_lambda`) is **L2 shrinkage on leaf weights**, pulling scores toward zero.
- (`α`/`reg_alpha` adds an optional L1 term `α Σ|w_j|`, omitted above for clarity.)

1.3 Additive (stage-wise) training

Trees cannot be optimized jointly in Euclidean space, so XGBoost adds one tree at a time. Writing the prediction at boosting step `t` as `ŷᵢ^(t)`: readthedocs

$$\hat{y}_i^{(0)} = \text{base\_score}, \qquad \hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

At step `t` we choose the tree `f_t` that most reduces:

$$\text{obj}^{(t)} = \sum_{i=1}^{n} l\Big(y_i,\ \hat{y}_i^{(t-1)} + f_t(x_i)\Big) + \Omega(f_t) + \text{constant}$$

1.4 The second-order Taylor expansion (the heart of XGBoost)

For general losses we approximate `l` around the current prediction using a Taylor expansion to **second order**:

$$\text{obj}^{(t)} \approx \sum_{i=1}^{n} \Big[l(y_i, \hat{y}_i^{(t-1)}) + g_i\, f_t(x_i) + \tfrac{1}{2} h_i\, f_t^2(x_i) \Big] + \Omega(f_t)$$

where the **gradient** and **Hessian** of the loss w.r.t. the previous prediction are:

$$g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)}), \qquad h_i = \partial^2_{\hat{y}^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)})$$

Dropping constants, the objective for the new tree depends on the data **only through** g_i **and** h_i:

$$\tilde{\text{obj}}^{(t)} = \sum_{i=1}^{n} \left[g_i \, f_t(x_i) + \tfrac{1}{2} h_i \, f_t^2(x_i) \right] + \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

This is why XGBoost can support *any* twice-differentiable custom loss with a single solver: you only supply g_i and h_i. Using the second derivative (Newton's method in function space) rather than only the gradient is what distinguishes XGBoost's "Newton boosting" from Friedman's classic first-order gradient boosting.

1.5 Optimal leaf weight and structure score

Group samples by leaf: let $I_j = \{i : q(x_i) = j\}$, and define the leaf-aggregated statistics $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$. Because every point in a leaf gets the same score w_j: readthedocs

$$\tilde{\text{obj}}^{(t)} = \sum_{j=1}^{T} \left[G_j w_j + \tfrac{1}{2}(H_j + \lambda) w_j^2 \right] + \gamma T$$

Each leaf term is an independent quadratic in w_j. Setting the derivative to zero gives the **optimal leaf weight**: readthedocs

$$\boxed{w_j^* = -\frac{G_j}{H_j + \lambda}}$$

Substituting back yields the **structure score** — a measure of how good a fixed tree shape is (lower is better): readthedocs

$$\boxed{\text{obj}^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T}$$

Note how λ in the denominator both shrinks leaf weights and regularizes the score, while γ charges a flat cost per leaf.

1.6 The split-gain formula

Since we cannot enumerate all tree structures, XGBoost grows greedily: for a candidate split of a node's instance set into left (L) and right (R) children, the improvement in the objective is the parent's score minus the children's scores, minus the cost of the extra leaf:

$$\boxed{\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma}$$

Interpretation: the first two terms are the children's fitness, the third is the un-split parent's fitness, and γ is the "toll" for adding a leaf. **A split is only kept if `Gain > 0`**, i.e. the fitness improvement exceeds γ. This is exactly how `gamma` acts as pre-pruning, and why λ makes splits more conservative (it damps every term).

1.7 The exact greedy split-finding algorithm

For each node:

1. For each feature, sort the instances by feature value.
2. Scan left-to-right, incrementally accumulating `G_L`, `H_L` (and deducing `G_R = G − G_L`, `H_R = H − H_L`).
3. Evaluate `Gain` at each candidate threshold; keep the best split across all features.
4. Recurse until a stopping condition (`max_depth`, `min_child_weight`, non-positive gain, etc.).

A single left-to-right scan suffices to evaluate every threshold for a feature — that is the practical payoff of the additive-statistics formulation. readthedocs

1.8 Approximate & histogram algorithms + the weighted quantile sketch

Exact greedy is `O(#instances × #features)` per level and does not scale. The **approximate algorithm** proposes a limited set of candidate split points per feature (bin boundaries), buckets instances into those bins, aggregates G/H per bin, and evaluates gain only at bin edges. Candidates can be proposed **globally** (once per tree) or **locally** (re-proposed per node). APXML

To choose bin boundaries well, XGBoost uses a **weighted quantile sketch**. Define the rank function for feature k:

$$r_k(z) = \frac{1}{\sum_{(x,h)\in\mathcal{D}_k} h} \sum_{(x,h)\in\mathcal{D}_k,\, x<z} h$$

and choose candidate points `{s_{k1},…,s_{kl}}` so that consecutive ranks differ by less than an approximation factor `ε`: `|r_k(s_{k,j}) − r_k(s_{k,j+1})| < ε`. This yields roughly `1/ε` candidates. The crucial subtlety: **each instance is weighted by its Hessian `hᵢ`**, not by count. This is justified by rewriting the objective as a *weighted squared error*: arxiv

$$\sum_i \tfrac{1}{2} h_i \left(f_t(x_i) - g_i/h_i \right)^2 + \Omega(f_t) + \text{const}$$

i.e. fitting label `gᵢ/hᵢ` with weight `hᵢ`. Because no existing quantile sketch handled weighted data with provable error bounds, the XGBoost authors introduced a **distributed weighted quantile sketch** with theoretical guarantees (a core contribution of the 2016 paper). arxiv

The **`hist`** **tree method** (the default since XGBoost 2.0) builds a global integer histogram of bins once and reuses it, making it the fastest built-in algorithm. Its bin count is controlled by **`max_bin`**, **which defaults to 256** (per the XGBoost source, `DMLC_DECLARE_FIELD(max_bin).set_lower_bound(2).set_default(256)`, and the official Tree Methods documentation). Higher `max_bin` = more split candidates = potentially higher accuracy but slower. GitHub Medium

1.9 Sparsity-aware split finding (missing values)

Real data is sparse (missing values, one-hot zeros). XGBoost learns a **default direction** at every node: during split finding it tries sending all missing/absent instances left, then right, and keeps whichever gives higher gain. Only the *non-missing* entries are enumerated (`Iₖ`), so the algorithm's cost is linear in the number of present values. The authors report this sparsity-aware variant ran **~50× faster** than the naïve dense version on the Allstate-10K dataset. Practically, this means **you should pass `NaN`s directly** — XGBoost handles them natively and often better than imputation. ACM SIGKDD

1.10 Shrinkage and subsampling

Two extra overfitting controls beyond `Ω`:

- **Shrinkage (`eta`), learning rate):** each new tree's contribution is scaled: `ŷ^(t) = ŷ^(t−1) + η·f_t(x)`. Smaller `η` leaves "room" for future trees (Friedman's idea) — more robust but requires more trees.
- **Subsampling: row** subsampling (`subsample`) uses a random fraction of rows per boosting round; **column** subsampling (`colsample_bytree`/`bylevel`/`bynode`) uses random feature subsets. Both inject randomness that reduces variance, and column subsampling additionally speeds up training. XGBoost XGBoost Documentation

1.11 Worked numerical example — gradients & Hessians

Squared-error loss `l = ½(y − ŷ)²`: `gᵢ = ŷ − y`, `hᵢ = 1`. Suppose current prediction `ŷ = 2.0`

for a sample with `y = 5.0`: `g = 2.0 − 5.0 = −3.0`, `h = 1`. A single-leaf weight (with `λ=1`) would be `w* = −(−3.0)/(1+1) = 1.5`, nudging the prediction up toward 5. XGBoosting

Logistic loss (binary): with `p = σ(ŷ) = 1/(1+e^{−ŷ})`, `gᵢ = p − y`, `hᵢ = p(1−p)`. Suppose `ŷ = 0` so `p = 0.5`, and true label `y = 1`: `g = 0.5 − 1 = −0.5`, `h = 0.5·0.5 = 0.25`. Leaf weight (`λ=1`): `w* = −(−0.5)/(0.25+1) = 0.4`, pushing the log-odds up (toward class 1). These formulas are confirmed by the DMLC GPU paper (arXiv:1806.11248), which states the logistic gradient/Hessian as `gᵢ = σ(ŷᵢ) − yᵢ` and `hᵢ = σ(ŷᵢ)(1 − σ(ŷᵢ))`. arxiv

Poisson loss (log link, `λ_rate = e^{ŷ}`): `gᵢ = e^{ŷ} − y`, `hᵢ = e^{ŷ}`. (In XGBoost's implementation the Hessian is computed as `exp(ŷ + max_delta_step)` — with `max_delta_step` defaulting to 0.7 for Poisson — as a numerical safeguard; the mathematically pure Hessian is `e^{ŷ}`.) arxiv + 3

1.12 Worked numerical example — computing a split gain by hand

Consider a node with 4 samples. Using squared error, suppose the current gradients/Hessians are:

sample	feature x	g	h
A	1.0	−3.0	1
B	2.0	−1.0	1
C	3.0	+2.0	1
D	4.0	+4.0	1

Parent totals: `G = −3−1+2+4 = 2`, `H = 4`. Take `λ = 1`, `γ = 0`.

Candidate split "x < 2.5" → Left = {A,B}, Right = {C,D}:

- `G_L = −4, H_L = 2`; `G_R = 6, H_R = 2`.
- Left term: `(−4)²/(2+1) = 16/3 = 5.333`
- Right term: `6²/(2+1) = 36/3 = 12.0`
- Parent term: `2²/(4+1) = 4/5 = 0.8`
- `Gain = ½(5.333 + 12.0 − 0.8) − 0 = ½(16.533) = 8.267`

Because `Gain = 8.27 > 0`, the split is kept. If we had set `γ = 9`, the gain would be `8.27 − 9 < 0` and **no split** would be made — a concrete demonstration of `gamma` as pre-pruning. The resulting leaf weights would be `w*_L = −(−4)/(2+1) = 1.333` and `w*_R = −6/(2+1) = −2.0`.

Part 2 — The Learning Process, Round by Round

Training a gbtree booster proceeds as follows:

Step 0 — Initialization (`base_score` / intercept). Predictions start from a global bias. **Since XGBoost 2.0, `base_score` is automatically estimated** from the training labels (e.g., the mean label for squared error, or the log-odds of the positive rate for logistic) rather than the old hard-coded `0.5`. This gives a better starting point and faster convergence. You can still override it (`set_params(base_score=0.5)`), and `base_margin` lets you provide a per-sample bias (useful for stacking). XGBoost + 3

Step 1 — Compute g_i, h_i for every training instance using the current predictions and the chosen `objective`.

Step 2 — Build one tree to fit (g_i, h_i), using the split-gain formula from Part 1. Growth policy:

- **`grow_policy="depthwise"` (default):** split nodes closest to the root first, growing level by level. More conservative, less prone to overfitting — XGBoost's traditional style. GitHub
- **`grow_policy="lossguide"`:** split the node with the highest loss reduction first (leaf-wise), like LightGBM. Can be more accurate but overfits more easily; enables `max_leaves` and `max_depth=0` (unlimited depth) with the `hist` method. Constraints applied during growth: `max_depth`, `max_leaves`, `min_child_weight` (a leaf needs total Hessian ⩾ this), `gamma` (split needs gain > this), `min_split_loss`, and column/row subsampling. GitHub readthedocs

Step 3 — Apply shrinkage. Multiply the new tree's leaf weights by `eta` before adding.

Step 4 — Update predictions: $\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \cdot f_t(x)$.

Step 5 — Evaluate & check stopping. If an `eval_set` is provided, compute `eval_metric` on it. Repeat Steps 1–5 until either:

- `n_estimators` / `num_boost_round` rounds are reached, or
- **early stopping** triggers: if the validation metric has not improved for `early_stopping_rounds` consecutive rounds, stop and (in the sklearn API) automatically use the best iteration for prediction. MLJAR XGBoost Documentation

How each hyperparameter shapes this loop:

- `eta` ↓ → each round contributes less → need more rounds, but better generalization (the canonical trade-off: low `eta` + many trees + early stopping).
- `max_depth` ↑ / `max_leaves` ↑ → more expressive trees → lower bias, higher variance.
- `min_child_weight` ↑, `gamma` ↑, `lambda`/`alpha` ↑ → more conservative splits → higher bias, lower variance.

- `subsample` < 1, `colsample_*` < 1 → stochastic training → lower variance, faster.
- `n_estimators` ↑ → more capacity; pair with early stopping so you never manually guess the right number.

Part 3 — All Parameters: Meaning, Impact, Math, and When to Tune

Defaults below are for the tree booster unless noted. `[alias]` *gives the scikit-learn name.*

3.1 General parameters

Parameter	Default	Meaning / impact
`booster`	`gbtree`	`gbtree` (trees), `gblinear` (**deprecated since 3.3.0**, linear models), `dart` (trees + dropout). Use `gbtree` unless you have a specific reason.
`device`	`cpu`	`cpu` or `cuda`/`cuda:0`. Replaces the old `gpu_id`. Set `device="cuda"` + `tree_method="hist"` for GPU training. Old aliases `gpu_hist`, `gpu_id` were **removed in 3.1**. XGBoost Documentation
`tree_method`	`auto`→`hist`	`hist` (default since 2.0, histogram/fastest), `exact` (enumerate all splits, small data only), `approx` (quantile sketch per iteration), `auto` readthedocs (chooses for you). XGBoosting XGBoost Documentation
`nthread` / `n_jobs`	all cores	Number of CPU threads. Set to 1 for XGBoost when nesting inside sklearn's `GridSearchCV` to avoid thread thrashing. XGBoost Documentation
`verbosity`	1	0 (silent) – 3 (debug).

3.2 Tree-booster parameters

Parameter `[alias]`	Default	Range	Role & math	When to tune
`eta` `[learning_rate]`	0.3	(0,1]	Shrinks each tree: `ŷ+=η·f`. readthedocs Lower→more robust, needs more trees.	Almost always; set 0.01–0.1 for final models.

Parameter [alias]	Default	Range	Role & math	When to tune
gamma [min_split_loss]	0	[0,∞)	Min gain to split; the $-\gamma$ term in Gain. Higher→fewer splits. XGBoost readthedocs	When overfitting despite depth control.
max_depth	6	[0,∞)	Max tree depth. Dominant complexity knob; deeper→more interactions, more overfit. XGBoost readthedocs	First structural param to tune (try 3–10).
min_child_weight	1	[0,∞)	Min sum of Hessians H_j in a child. readthedocs For squared error = min #samples; for logistic ≈ $\Sigma p(1-p)$. Higher→conservative. XGBoost	Tune with max_depth.
max_delta_step	0	[0,∞)	Caps each leaf's weight update magnitude. Helps extreme class imbalance in logistic. readthedocs	Rare; try 1–10 for very imbalanced data.
subsample	1	(0,1]	Row fraction per round (once per iteration). readthedocs <1 reduces variance.	Set 0.6–0.9 to regularize/speed up.
colsample_bytree	1	(0,1]	Feature fraction per tree. readthedocs	0.6–0.9 for many features.
colsample_bylevel	1	(0,1]	Feature fraction per level. readthedocs	Fine-grained regularization.

Parameter [alias]	Default	Range	Role & math	When to tune
colsample_bynode	1	(0,1]	Feature fraction per split readthedocs (this is the random-forest-style knob).	High-dim/correlated features.
lambda [reg_lambda]	1	[0,∞)	L2 on leaf weights; the +λ in w*=−G/(H+λ). readthedocs Smooths weights.	Increase to reduce overfitting.
alpha [reg_alpha]	0	[0,∞)	L1 on leaf weights; induces sparsity. XGBoost readthedocs	High-dim, feature-selection-like effect.
scale_pos_weight	1	(0,∞)	Multiplies positive-class gradients; balances classes. Set ≈ #neg/#pos. readthedocs	Imbalanced binary classification.
grow_policy	depthwise	—	depthwise (level-by-level) or lossguide (leaf-wise). readthedocs	lossguide for speed/accuracy on large data.
max_leaves	0	[0,∞)	Max leaves (only meaningful with lossguide; 0 = no limit). readthedocs	With lossguide.
max_bin	256	[2,∞)	Histogram bins for hist/approx. More→finer splits, slower. readthedocs	Raise (e.g. 512) for accuracy; lower for speed.
monotone_constraints	none	—	Force feature→prediction monotonicity, readthedocs e.g. (1,0,-1). Enforced by rejecting violating splits.	Regulated/known-direction features (credit, pricing).

Parameter `[alias]`	Default	Range	Role & math	When to tune
`interaction_constraints`	none	—	Restrict which features may interact within a tree, e.g. `[[0,1],[2,3]]`.	Domain knowledge / interpretability.
`num_parallel_tree`	1	—	Trees per round; >1 gives boosted random forests. readthedocs	Rarely.

3.3 DART parameters (`booster="dart"`)

DART = "Dropouts meet Multiple Additive Regression Trees" — it randomly drops existing trees during each boosting round to prevent later trees from dominating (over-specialization).

Parameter	Default	Meaning
`rate_drop`	0.0	Fraction of trees dropped per round (dropout rate).
`skip_drop`	0.0	Probability of skipping dropout entirely in a round XGBoosting (higher priority than `rate_drop`). XGBoost
`sample_type`	`uniform`	How dropped trees are chosen: `uniform` or `weighted`. readthedocs
`normalize_type`	`tree`	How new trees are scaled after dropout: `tree` or `forest`. readthedocs
`one_drop`	0	If 1, at least one tree is always dropped (enables epsilon-dropout).

DART inherits all `gbtree` params (`eta`, `max_depth`, …). It trades training/prediction speed for potential accuracy gains. Dmlc

3.4 Linear-booster parameters (`booster="gblinear"`, deprecated since 3.3.0)

The v3.3.0 docs state verbatim: *"Deprecated since version 3.3.0: booster=gblinear is deprecated and support will be removed in a future release."* For completeness: XGBoost

- `updater` (default `shotgun`): `shotgun` = parallel "hogwild" coordinate descent (non-deterministic); `coord_descent` = ordinary, deterministic coordinate descent. XGBoost Documentation
- `feature_selector` (default `cyclic`): `cyclic`, `shuffle` (use with `shotgun`); `random`, `greedy`, `thrifty` (use with `coord_descent`). `greedy` selects the coordinate with the

largest gradient magnitude. HexDocs HexDocs

- `top_k` (default 0 = all): restrict `greedy`/`thrifty` to the top-k features. HexDocs
- **Important gotcha:** for gblinear, `lambda` and `alpha` **both default to 0 and are normalized by the number of training examples** — different from the tree booster where `lambda` defaults to 1 and is un-normalized. XGBoost XGBoost

3.5 Learning-task parameters

`objective` — defines the loss (hence g_i, h_i):

Objective	Task	g_i, h_i (per instance)
`reg:squarederror`	regression (default)	$g = \hat{y} - y$, $h = 1$
`reg:logistic`	probability regression	$g = p - y$, $h = p(1-p)$, $p=\sigma(\hat{y})$
`binary:logistic`	binary classification (prob. output)	same as above
`binary:logitraw`	binary, raw score output	logistic g/h
`count:poisson`	count data, log link	$g = e^{\hat{y}} - y$, $h = e^{\hat{y}}$ (impl. uses $e^{\hat{y}+\text{max\_delta\_step}}$)
`reg:gamma`	positive skewed (insurance severity), log link XGBoost	$g = 1 - y\cdot e^{-\hat{y}}$, $h = y\cdot e^{-\hat{y}}$ arxiv
`reg:tweedie`	insurance total loss, log link	$g = -y\cdot e^{(1-\rho)\hat{y}}+e^{(2-\rho)\hat{y}}$ XGBoost (ρ=`tweedie_variance_power`)
`multi:softmax`	multiclass (class label); needs `num_class` XGBoost	softmax cross-entropy (diagonal Hessian bound) XGBoost Documentation
`multi:softprob`	multiclass (probability matrix)	same
`rank:ndcg` / `rank:map` / `rank:pairwise`	learning to rank (LambdaMART) XGBoost Documentation	pairwise/listwise
`survival:cox` / `survival:aft`	survival analysis	Cox partial likelihood / AFT

`eval_metric` — the monitoring metric (default follows `objective`): `rmse`, `mae`, `rmsle`, `logloss`, `error` (0/1), `auc`, `aucpr`, `merror`, `mlogloss`, `ndcg`, `map`, `poisson-nloglik`, `gamma-nloglik`, `tweedie-nloglik`, `cox-nloglik`, `aft-nloglik`. If multiple are given, the **last** one drives early stopping. XGBoost Documentation XGBoost Documentation

Other task params: `base_score` (intercept, auto-estimated since 2.0), `seed`/`random_state` (reproducibility), `num_class` (multiclass), `tweedie_variance_power` ($1<\rho<2$).

3.6 Categorical parameters

`enable_categorical=True` turns on native categorical handling (stable since 3.0; a re-coder added in 3.1 keeps string categories consistent at inference). `max_cat_to_onehot` decides per-feature whether to use one-hot or optimal partition-based splits. XGBoost Documentation + 2

Part 4 — A Complete, Commented End-to-End Example

This uses the scikit-learn **Breast Cancer Wisconsin** dataset (binary classification), showing **both APIs**, early stopping, feature importance, SHAP, `xgb.cv`, Optuna, and model persistence.

python

```python
# ============================================================
# 0. Imports  (XGBoost 2.x/3.x, scikit-learn, shap, optuna)
# ============================================================
import numpy as np
import pandas as pd
import xgboost as xgb
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.metrics import (accuracy_score, roc_auc_score,
                             average_precision_score, classification_report)
import matplotlib.pyplot as plt

print("XGBoost version:", xgb.__version__)   # e.g. 3.3.0

# ============================================================
# 1. Load & explore data
# ============================================================
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target        # 569 samples, 30 features; y=1 benign
print(X.shape, "class balance:", np.bincount(y))   # slight imbalance
print(X.describe().T.head())          # quick numeric summary

# ============================================================
# 2. Train / validation / test split (stratified)
# ============================================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
# Carve a validation set out of train for early stopping
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train, y_train, test_size=0.2, stratify=y_train, random_state=42)

# ============================================================
# 3A. scikit-learn API (recommended for most users)
#     early_stopping_rounds & eval_metric are CONSTRUCTOR args
#     in 2.x/3.x (they were moved out of fit()).
# ============================================================
clf = xgb.XGBClassifier(
    n_estimators=2000,          # upper bound; early stopping finds the real number
    learning_rate=0.05,
    max_depth=4,
    min_child_weight=2,
    subsample=0.8,
    colsample_bytree=0.8,
```

```
    reg_lambda=1.0,
    reg_alpha=0.0,
    objective="binary:logistic",
    eval_metric="aucpr",        # AUC-PR: good for mild imbalance
    tree_method="hist",         # default since 2.0; add device="cuda" for GPU
    early_stopping_rounds=50,   # constructor arg in modern XGBoost
    random_state=42,
)
clf.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=100)
print("Best iteration:", clf.best_iteration)   # trees actually used

# Prediction automatically uses the best iteration when early stopping is on
proba = clf.predict_proba(X_test)[:, 1]
pred  = clf.predict(X_test)
print("Test ACC :", accuracy_score(y_test, pred))
print("Test AUC :", roc_auc_score(y_test, proba))
print("Test AUCPR:", average_precision_score(y_test, proba))
print(classification_report(y_test, pred))

# ============================================================
# 3B. Native (Booster) API with DMatrix — full control
# ============================================================
dtrain = xgb.DMatrix(X_tr, label=y_tr)
dval   = xgb.DMatrix(X_val, label=y_val)
dtest  = xgb.DMatrix(X_test, label=y_test)

params = dict(objective="binary:logistic", eval_metric=["logloss", "aucpr"],
              eta=0.05, max_depth=4, min_child_weight=2,
              subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0,
              tree_method="hist", seed=42)

booster = xgb.train(
    params, dtrain, num_boost_round=2000,
    evals=[(dtrain, "train"), (dval, "valid")],
    early_stopping_rounds=50, verbose_eval=100,
)
# With the native API, slice to the best iteration explicitly:
best_range = (0, booster.best_iteration + 1)
native_proba = booster.predict(dtest, iteration_range=best_range)

# ============================================================
# 4. Cross-validation with xgb.cv (native API)
# ============================================================
cv = xgb.cv(
```

```
    params, dtrain, num_boost_round=2000, nfold=5, stratified=True,
    early_stopping_rounds=50, metrics="aucpr", seed=42, as_pandas=True)
print("CV best rounds:", len(cv), "| CV test AUCPR:",
      cv["test-aucpr-mean"].iloc[-1], "+/-", cv["test-aucpr-std"].iloc[-1])

# ============================================================
# 5. Feature importance: gain, weight, cover  (interpret carefully!)
# ============================================================
for imp in ("gain", "weight", "cover"):
    fig, ax = plt.subplots(figsize=(6, 8))
    xgb.plot_importance(clf.get_booster(), importance_type=imp,
                        max_num_features=10, ax=ax, title=f"Importance ({imp})")
    plt.tight_layout(); plt.savefig(f"importance_{imp}.png")

# ============================================================
# 6. SHAP values — the preferred, consistent interpretation
# ============================================================
import shap
explainer = shap.TreeExplainer(clf)          # exact TreeSHAP for trees
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test, show=False)   # global importance + direction
plt.tight_layout(); plt.savefig("shap_summary.png")
# You can also compute SHAP natively:
#   contribs = booster.predict(dtest, pred_contribs=True)

# ============================================================
# 7. Hyperparameter tuning with Optuna (Bayesian / TPE)
# ============================================================
import optuna

def objective(trial):
    param = {
        "objective": "binary:logistic", "eval_metric": "aucpr",
        "tree_method": "hist", "random_state": 42,
        "n_estimators": 3000,
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3, log=True),
        "max_depth": trial.suggest_int("max_depth", 2, 8),
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 20),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
        "gamma": trial.suggest_float("gamma", 1e-8, 5.0, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-3, 10.0, log=True),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-8, 10.0, log=True),
        "early_stopping_rounds": 50,
```

```
    }
    model = xgb.XGBClassifier(**param)
    model.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=False)
    p = model.predict_proba(X_val)[:, 1]
    return average_precision_score(y_val, p)   # maximize AUC-PR

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50, show_progress_bar=True)
print("Best params:", study.best_params, "| Best AUCPR:", study.best_value)

# ============================================================
# 8. Save & load  (use JSON/UBJSON — stable, portable format)
# ============================================================
clf.save_model("bc_model.json")              # sklearn wrapper
booster.save_model("bc_booster.ubj")         # native, UBJSON (compact/stable)

loaded = xgb.XGBClassifier()
loaded.load_model("bc_model.json")
assert np.allclose(loaded.predict_proba(X_test)[:, 1], proba)
print("Model round-trip OK.")
```

Notes tied to current API: (1) `early_stopping_rounds` and `eval_metric` belong in the **constructor** in modern XGBoost (moved out of `fit()`); alternatively use `xgb.callback.EarlyStopping(rounds=..., save_best=True)`. (2) With the sklearn API, prediction auto-uses `best_iteration`; with the native API you must pass `iteration_range`. (3) Prefer JSON/UBJSON model files over the removed legacy binary format (binary serialization was removed in 3.1). XGBoost Documentation

---

Part 5 — Best Practices with Examples

1. Tune in the right order (biggest levers first). A widely used, efficient recipe: (a) fix a moderate `learning_rate` (0.1) and a large `n_estimators` with early stopping to find the right tree count; (b) tune **tree structure** (`max_depth`, `min_child_weight`, `gamma`); (c) tune **sampling** (`subsample`, `colsample_bytree`); (d) tune **regularization** (`reg_lambda`, `reg_alpha`); (e) finally **lower** `eta` (0.01–0.05) and **raise** `n_estimators` for the final model. A useful shortcut proven in practice (Random Realizations): searching tree params at a *high* learning rate recovers essentially the same optimal tree params as at a low rate — so tune structure fast, then drop `eta` at the end.

2. Use early stopping properly. Always hold out a validation set (never the test set), set `n_estimators` high (1000–5000), and let `early_stopping_rounds` (a common heuristic: ~10% of total rounds, e.g. 50 with 1000 trees) find the stopping point. This automates the single most important anti-overfitting decision. MLJAR

3. Handle imbalance with `scale_pos_weight` + the right metric. Set `scale_pos_weight ≈ sum(negatives)/sum(positives)`. Monitor **AUC-PR (`aucpr`)** or F1, *not* accuracy — a 99%-majority dataset scores 99% accuracy while detecting zero minorities. If you need *calibrated probabilities*, be cautious: `scale_pos_weight` distorts them, so consider leaving it at 1 and thresholding instead. `max_delta_step` (1–10) can stabilize updates under extreme imbalance.
XGBoosting + 3

```python
spw = (y_train == 0).sum() / (y_train == 1).sum()
clf = xgb.XGBClassifier(scale_pos_weight=spw, eval_metric="aucpr", tree_method="hist"
```

4. Let XGBoost handle missing values natively. Don't impute reflexively — pass `NaN`s through. Sparsity-aware split finding learns a default direction per node and is both faster and often more accurate than mean/median imputation.

5. Prefer native categorical handling for high-cardinality features. Set the column dtype to `category` and pass `enable_categorical=True`. Partition-based splits (`value ∈ categories`) are more memory- and compute-efficient than one-hot, avoid the dimensionality blow-up of one-hot for ZIP-code-like features, and (per practitioners) often give better accuracy. One-hot is fine for low-cardinality features.

6. Prevent overfitting with layered defenses: shallower `max_depth`, higher `min_child_weight`/`gamma`, `reg_lambda`/`reg_alpha`, `subsample`/`colsample_* < 1`, lower `eta` + early stopping. Watch train-vs-validation learning curves (`booster.evals_result()` / `plot_metric`); a widening gap = overfitting.

7. Avoid data leakage. Fit all preprocessing (scaling, target encoding, imputation) **inside** cross-validation folds, never on the full dataset before splitting. Target-encode categoricals only on training folds. Keep the test set untouched until the very end.

8. GPU acceleration. Set `device="cuda"` + `tree_method="hist"` (the old `gpu_hist` was removed in 3.1). For huge data use `QuantileDMatrix` or the new `ExtMemQuantileDMatrix` (3.0) for external-memory training; 3.x can exploit NVLink-C2C for terabyte-scale GPU training.
XGBoost Documentation XGBoost Documentation

9. `hist` vs `exact` vs `approx`. Use `hist` (default) for essentially everything — it's the fastest and scales well. Use `exact` only on small datasets where you want the theoretically optimal splits. `approx` (per-iteration sketch) can occasionally edge out `hist` in accuracy for non-constant-Hessian objectives, but `hist` with a higher `max_bin` usually matches it faster.
XGBoost Documentation + 2

10. Interpret importance correctly — prefer SHAP. The three built-in importances answer different questions: **`gain`** = average loss reduction from a feature's splits (best default for "which

features matter for accuracy"), `weight`/**frequency** = how often a feature is used (biased toward high-cardinality/continuous features), `cover` = how many samples its splits touch. All three can disagree and are known to be inconsistent (Lundberg's work). **SHAP (TreeExplainer)** gives consistent, theoretically grounded, per-prediction attributions with direction (positive/negative), and is the recommended tool for both global and local interpretation. Medium + 3

11. Use Bayesian optimization (Optuna) over grid search. For >2–3 hyperparameters, Optuna's TPE sampler finds better configs faster than grid/random search, and its XGBoost pruning integration kills unpromising trials early. Use log-scale ranges for `learning_rate`, `reg_lambda`, `reg_alpha`, `gamma`.

12. XGBoost vs LightGBM vs CatBoost vs neural nets.

- **XGBoost:** the most reliable, tunable, best-documented default for tabular data; depth-wise growth is robust; huge ecosystem. Best all-round choice and still a dominant Kaggle winner on structured data.
- **LightGBM:** materially faster on large/sparse data (leaf-wise growth, GOSS, EFB). The LightGBM authors report per-iteration speedups of **~2×–9.4×** over XGBoost's histogram mode, and note XGBoost ran out of memory on their largest sparse benchmarks (KDD10/KDD12) while LightGBM trained normally. As a concrete production example, Microsoft's Bing Ads reportedly retrains click-prediction models "every 30 minutes on 500 million ad impressions — a task that previously took 6 hours with XGBoost." Trade-off: leaf-wise growth overfits more easily and needs careful tuning. XGBoosting + 2
- **CatBoost:** best out-of-the-box for datasets dominated by categorical features (ordered boosting, symmetric/oblivious trees, minimal preprocessing); often strong with little tuning.
- **Neural nets:** prefer when you have very large data with strong feature interactions, unstructured inputs (text/image/audio), or need end-to-end representation learning. For most small-to-medium tabular problems, gradient-boosted trees still beat deep nets.

13. Common pitfalls: forgetting that `early_stopping_rounds` moved to the constructor; using the native API without `iteration_range` (so you predict with *all* trees, not the best); reading `weight` importance as ground truth; tuning/selecting features on the full dataset (leakage); setting both XGBoost and sklearn to all threads (thread thrashing — set XGBoost `n_jobs=None`, sklearn `n_jobs=1` when nesting). XGBoost Documentation

---

Part 6 — Useful Material to Dive Deeper (Verified References)

Primary papers

- **Chen & Guestrin (2016), "XGBoost: A Scalable Tree Boosting System," KDD.** The foundational paper — regularized objective, weighted quantile sketch, sparsity-aware split

finding, system design. arXiv: `https://arxiv.org/abs/1603.02754` (PDF: `https://arxiv.org/pdf/1603.02754`); DOI `10.1145/2939672.2939785`.

- **Friedman (2001), "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics* 29(5):1189–1232.** Origin of gradient boosting and shrinkage. DOI `10.1214/aos/1013203451`. Project Euclid
- **Friedman (2002), "Stochastic Gradient Boosting," *Computational Statistics & Data Analysis* 38(4):367–378.** Row subsampling. DOI `10.1016/S0167-9473(01)00065-2`. arxiv
- **Vinayak & Gilad-Bachrach (2015), "DART: Dropouts meet Multiple Additive Regression Trees," AISTATS.** The DART booster.
- **Lundberg & Lee (2017), "A Unified Approach to Interpreting Model Predictions," NeurIPS 30.** The SHAP framework. PDF: `https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf`. See also Lundberg et al. (2020) on TreeSHAP. NeurIPS

Official documentation (all current for 3.3.x)

- Docs home: `https://xgboost.readthedocs.io/`
- **"Introduction to Boosted Trees"** (the canonical math tutorial): `https://xgboost.readthedocs.io/en/stable/tutorials/model.html`
- Parameters reference: `https://xgboost.readthedocs.io/en/stable/parameter.html`
- Tree Methods: `https://xgboost.readthedocs.io/en/stable/treemethod.html`
- Categorical data, DART, Monotonic/Interaction Constraints, Intercept, GPU, Python API — all under the same tutorials tree.
- GitHub: `https://github.com/dmlc/xgboost` (issues, source, release notes).

Textbook

- Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning* (2nd ed., 2009), **Ch. 10 "Boosting and Additive Trees."** Free PDF from the authors' Stanford page. The definitive theoretical grounding.

Tutorials, courses, tooling

- **StatQuest with Josh Starmer** — the four-part XGBoost video series (Math Details, Regression, Classification, Optimizations) on YouTube; the clearest visual walk-through of the gain/leaf-weight math.
- **Machine Learning Mastery** — practical guides, incl. "How to Configure XGBoost for Imbalanced Classification": `https://machinelearningmastery.com/xgboost-for-imbalanced-classification/`.
- **Optuna** — hyperparameter optimization: `https://optuna.org/` (docs `https://optuna.readthedocs.io/`), with a first-class XGBoost pruning integration.

- **SHAP** — `https://shap.readthedocs.io/` and Lundberg's "Interpretable Machine Learning with XGBoost" (TDS).
- **Kaggle** — competition notebooks/discussions remain the best source of applied XGBoost tricks for tabular data.
- **XGBoosting.com** and **Random Realizations** ("The Ultimate Guide to XGBoost Parameter Tuning with Optuna") — high-quality, code-first parameter and tuning references.

Caveats

- **Version currency.** This guide reflects **XGBoost 3.3.0 (released 17 June 2026)**, confirmed via PyPI (`xgboost-3.3.0.tar.gz`, uploaded Jun 17 2026) and the readthedocs release notes. Key modern-API points: `hist` is the default `tree_method` since 2.0; `base_score` is auto-estimated since 2.0; `gpu_hist`/`gpu_id` were removed in 3.1 (use `device="cuda"`); the legacy binary model format was removed in 3.1 (use JSON/UBJSON); `gblinear` is deprecated as of 3.3.0. Verify against the docs for your installed version, since defaults can change across releases. PyPI + 2
- **Objective-specific gradient/Hessian formulas** were verified against the DMLC GPU paper (arXiv:1806.11248, logistic), the XGBoost source (`regression_obj.cu`, Poisson/Tweedie), and Sigrist (arXiv:1808.03064, Poisson/Gamma). The `reg:gamma` formula is the γ=1 specialization of Sigrist's general result (high-confidence, derived rather than quoted from a verbatim code line). XGBoost's Poisson Hessian uses a `max_delta_step` safeguard (`e^{ŷ+max_delta_step}`, default 0.7), a numerical detail beyond the pure math. Multiclass softmax uses a diagonal *upper bound* on the Hessian, not the full matrix. XGBoost Documentation
- **Comparative speed claims** (LightGBM 2×–9.4× faster; Bing Ads 6h→30min) come from the LightGBM authors' benchmarks and a secondary production write-up; relative performance is dataset-dependent and benchmarks can be cherry-picked. Treat them as directional, not guarantees — always benchmark on your own data.
- **Code** is written for the 2.x/3.x API and uses standard datasets; exact metric values will vary with random seeds, library versions, and hardware. The SHAP and Optuna snippets require those packages installed separately.